table. We'll rephrase that.

Rails doesn't work too well unless each table has a primary key that Rails can manage. By default, Rails will create numeric primary keys, but you can also use other types such as UUIDs, depending on what your actual database provides. Rails is less fussy about the name of the column. So for your average Rails application, our strong advice is to go with the flow and let Rails have its `id` column.

If you decide to be adventurous, you can start by using a different name for the primary key column (but keeping it as an incrementing integer). Do this by specifying a `:primary_key` option on the `create_table` call:

```
create_table :tickets, primary_key: :number do |t|
  t.text :description

  t.timestamps
end
```

This adds the `number` column to the table and sets it up as the primary key:

```
$ sqlite3 storage/development.sqlite3 ".schema tickets"
CREATE TABLE tickets ("number" INTEGER PRIMARY KEY AUTOINCREMENT
NOT NULL, "description" text DEFAULT NULL, "created_at" datetime
DEFAULT NULL, "updated_at" datetime DEFAULT NULL);
```

The next step in the adventure might be to create a primary key that isn't an integer. Here's a clue that the Rails developers don't think this is a good idea: migrations don't let you do this (at least not directly).

### Tables with No Primary Key

Sometimes we may need to define a table that has no primary key. The most common case in Rails is for *join tables*—tables with just two columns where each column is a foreign key to another table. To create a join table using migrations, we have to tell Rails not to automatically add an `id` column: